Chain of Thought vs. ReAct
Explained Simply, With Easy Examples

A plain-language guide to two ways an AI model can “think” before answering — reasoning alone versus reasoning combined with real lookups — with everyday examples for each.

Created: August 16, 2026 at 06:11 PM

Based on: Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models” (2022) and Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models” (ICLR 2023)

Index

1. Overview

When you ask an AI model a question, it can answer in different styles. Two popular ones are called **Chain of Thought** (CoT) and **ReAct** (short for Reason + Act). They sound similar and they're related, but they solve different problems. This guide explains both in plain words, with simple examples, so the difference is easy to remember.

The short version: **Chain of Thought is thinking out loud. ReAct is thinking out loud while also being able to check things and take action.** Everything else in this document builds on that one sentence.

2. What Is Chain of Thought (CoT)?

2.1 The Simple Idea

Normally, you might ask a model a question and it jumps straight to an answer. Chain of Thought just means asking the model to show its working — to break the problem into small steps and write each step down before giving the final answer, the same way a student is asked to “show your work” on a math test.

All of this thinking happens **inside the model's own head**. It doesn't look anything up, doesn't use a calculator or a search engine — it just organizes its own knowledge more carefully before answering.

2.2 Example: A Math Problem

Question: “A train leaves at 3:00 pm, travels for 2 hours, then stops for 30 minutes at a station. What time does it finish its journey?”

```
Thought: The train leaves at 3:00 pm and travels for 2 hours.
         3:00 pm + 2 hours = 5:00 pm.
Thought: Then it stops for 30 minutes.
         5:00 pm + 30 minutes = 5:30 pm.
Answer:  5:30 pm.
```

Notice everything needed to solve this — the times, the arithmetic — was already in the question. The model just had to reason carefully through it, step by step. That's a perfect job for Chain of Thought.

2.3 The Catch

Chain of Thought is great for reasoning and logic puzzles, but it has one weak spot: if the answer depends on a fact the model doesn't actually know — or a fact that changes over time, like today's weather, a stock price, or something that happened yesterday — CoT has no way to check. It can only reason over what it already believes, and if that belief is wrong or outdated, it will reason its way to a confident-sounding but wrong answer. This is often called **hallucination**.

3. What Is ReAct?

3.1 The Simple Idea

ReAct keeps the “thinking out loud” part of Chain of Thought, but adds one more ingredient: the ability to **pause and do something in the real world** — search the web, run a calculation, check a database — and then keep reasoning using what it just found out. It repeats a simple three-part cycle:

- **Thought** — what do I know, and what do I need to find out next?
- **Action** — go get that piece of information (search, calculate, look up).
- **Observation** — here's the real result — now think again.

This cycle can repeat as many times as needed before the model gives a final answer. The key difference from CoT: the reasoning is grounded in real, checked information instead of only the model's memory.

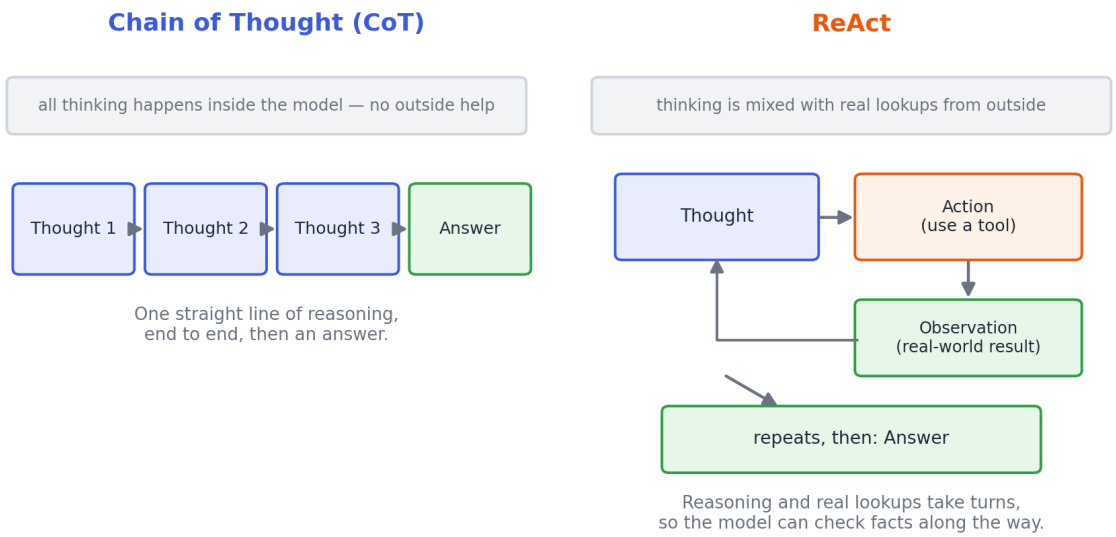

Figure 1. CoT reasons in one straight line from memory. ReAct alternates between reasoning and real lookups.

3.2 Example: The Weather Question

Question: “It's today — should I bring an umbrella in Paris?” This is exactly the kind of question CoT struggles with, because “today's weather” isn't something any model can simply know or reason out from memory.

```
Thought:     I need today's real weather in Paris — I can't
             know this from memory alone.
Action:      search[Paris weather today]
Observation: “Rainy, 15°C”
Thought:     It's rainy, so an umbrella is a good idea.
Answer:      Yes, bring an umbrella.
```

3.3 Why This Helps

The Action step is what makes the difference. Instead of guessing from general knowledge (“Paris is often mild”), the model checks today's actual forecast before answering. If the model had reasoned this one out

with plain Chain of Thought, it would have had to guess — and on a rainy day, that guess could easily be wrong.

4. Side-by-Side Comparison

4.1 Same Question, Two Approaches

Here is the umbrella question answered both ways, so you can see exactly where the two approaches diverge.

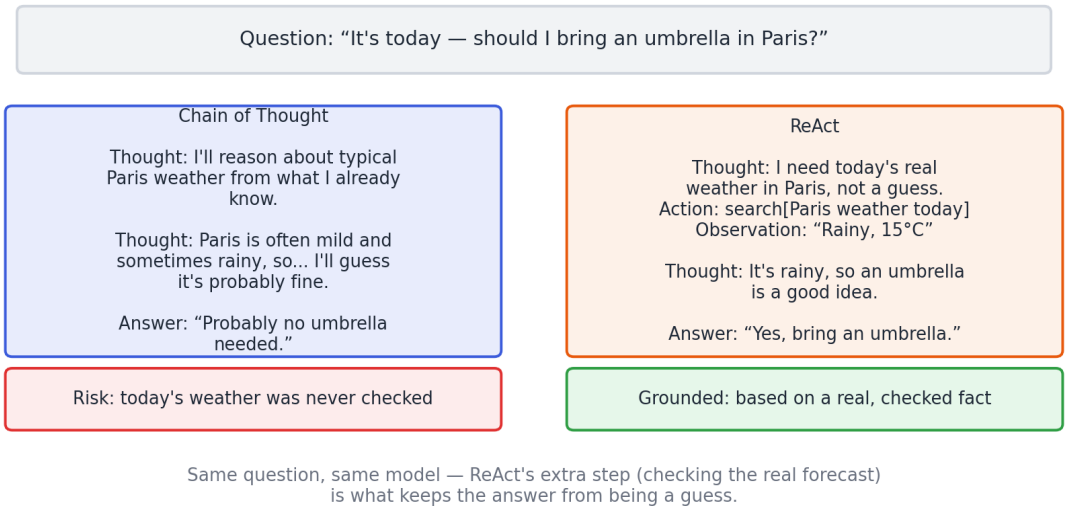

Figure 2. Same question, same model — CoT guesses from memory, ReAct checks a real source first.

4.2 Comparison Table

	Chain of Thought	ReAct
What it does	Reasons in steps, using only what it already knows	Reasons in steps, and can pause to look things up
Uses outside tools?	No	Yes (search, calculator, etc.)
Best for	Math, logic, word problems — anything solvable from the question itself	Questions needing current, specific, or outside facts
Main risk	Can sound confident while being factually wrong (hallucination)	Slower and more complex, since it may need several lookup steps

5. A Simple Analogy

Think about answering a trivia question at home.

- **Chain of Thought** is like answering entirely from memory, carefully thinking it through before you speak — but never leaving your chair.
- **ReAct** is like answering the same way, but if you're not sure of a fact, you're allowed to get up, check a book or your phone, come back, and continue thinking with that new information.

Both approaches think carefully. Only one of them can double-check itself against the real world.

6. In Code Terms (Python)

At a code level, the difference is about whether the loop ever calls out to something outside the model. A simplified sketch of each:

```
# Chain of Thought — one call, all reasoning happens in the response text
response = model.generate(
    prompt=f"{question}\nLet's think step by step:"
)
answer = response.text

# ReAct — a loop that can call real tools between reasoning steps
history = []
while True:
    step = model.generate(prompt=question, context=history)
    if step.action == "finish":
        answer = step.answer
        break
    result = run_tool(step.action, step.action_input)   # e.g. search()
    history.append((step.thought, step.action, result))  # feed it back in
```

The Chain of Thought version only ever calls the model. The ReAct version wraps that same kind of call in a loop that can also call `run_tool()` — that one addition is the entire mechanical difference between the two.

7. When to Use Which

Reach for Chain of Thought when the problem is self-contained — arithmetic, logic riddles, step-by-step explanations, planning within information you've already provided. There's nothing to look up, so extra tool calls would only slow things down.

Reach for ReAct when the question depends on something outside the model's own memory — current events, prices, live data, multi-step research questions, or anything where being factually correct matters more than being fast. The weather example in Section 3 is a small case of this; the same idea scales up to things like “what's this company's latest stock price and is it up or down this week?”

8. Conclusion

Chain of Thought and ReAct both make an AI model's reasoning more careful and more visible. The difference comes down to one thing: **ReAct can check the real world along the way, and Chain of Thought cannot.** That single capability is why ReAct tends to be more reliable on questions involving current or specific facts, while Chain of Thought remains a simple, fast, and perfectly good choice for problems that are pure reasoning from start to finish.

9. References

1. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* arXiv:2201.11903. https://arxiv.org/abs/2201.11903

2. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models.* International Conference on Learning Representations (ICLR 2023). arXiv:2210.03629. https://arxiv.org/abs/2210.03629

3. Examples, diagrams, and code sketches in this document were written for this document to illustrate the two concepts in plain language.